

Formalization of Crisp and Fuzzy Derivatives of Regular Expressions in Rocq

Bachelor's Thesis Defense

Mariam Katamashvili

Kutaisi International University
School of Computer Science

2026

Supervisors: Besik Dundua Furio Honsell

What this talk covers

- 1 Regular expressions and derivatives
- 2 Rocq
- 3 Automata
- 4 Fuzzy layer
- 5 Conclusion

Where we are

- 1 Regular expressions and derivatives
- 2 Rocq
- 3 Automata
- 4 Fuzzy layer
- 5 Conclusion

What are regular expressions and why do we need them?

We often need to describe a whole set of words by a single pattern. A regular expression is that pattern. It is a short piece of text that says which words match and which do not.

It is built from a few operators over an alphabet Σ :

$$r ::= \emptyset \mid \varepsilon \mid a \mid r+r \mid r \cdot r \mid r^* \mid r \& r \mid \neg r.$$

Where they are used:

- The **lexical analyzer** in every compiler splits source code into tokens.
- **Search** tools like `grep`, text editors, and log scanners find patterns in text.
- **Validation** checks that an email address or a phone number has the right shape.
- **Networks** scan traffic for known attack signatures.

How do we actually run a regular expression?

To check whether a word matches a pattern, we have to run the pattern on the word. There are two ways to do this.

Build a machine first

We turn the pattern into a separate automaton, then feed the word to that machine. This is the classical route (Thompson, Glushkov, McNaughton–Yamada).

Differentiate the pattern

(Brzozowski, 1964)

We never build a machine. The **derivative** $\mathcal{D}_a(r)$ is simply what is left to match after we read the symbol a . Taking one derivative per symbol already runs the pattern on the word.

Why this matters

Reading a symbol is the same as taking a derivative. If we keep derivatives in a **canonical form**, there are only finitely many of them. Each canonical derivative is a state of the automaton, so the automaton comes out on its own, with no extra construction.

What are derivatives used for?

Derivatives are used in real tools:

- A **lexer generator**, a tool that turns patterns into code that splits text into tokens, was built directly on derivatives for Standard ML by Owens, Reppy and Turon.
- Google's redgrep matcher compiles patterns to automata through **derivatives** instead of the usual NFA.
- Equivalence checkers decide whether two patterns match the same words by **comparing derivatives**.
- Verified regular-expression libraries in **Coq and Isabelle** (for example Krauss and Nipkow's) are built on derivatives because they are easy to reason about.

Two questions this thesis answers

- ① Does the theory still work when symbols match by **similarity** instead of exact equality?
- ② Can its core be **machine-checked** instead of trusted on paper?

Both answers are **yes**.

Where we are

- 1 Regular expressions and derivatives
- 2 Rocq
- 3 Automata
- 4 Fuzzy layer
- 5 Conclusion

What is Rocq?

Rocq (Coq) is an interactive proof assistant. It is built on a dependent type theory, the Calculus of Inductive Constructions, and it mechanically checks that every proof is valid.

- By the **Curry–Howard correspondence** a proposition is a type and a proof is a term of that type. Proving a statement means constructing such a term.
- The logic is **constructive**, so proofs carry computational content. A proof of an existence statement yields an explicit witness.
- Data and propositions are defined as **inductive types**, given by their constructors, which is what lets us reason by induction.
- Proofs are written with **tactics**, and the resulting term is verified by a small **kernel**.

Rocq in the real world

Rocq has checked a verified C compiler (CompCert), the Four Colour Theorem, and real cryptography code.

Where we are

- 1 Regular expressions and derivatives
- 2 Rocq
- 3 Automata**
- 4 Fuzzy layer
- 5 Conclusion

Regular expressions and languages

First we tell Rocq what a regular expression is. Each kind of expression becomes one case of a data type.

```
Inductive regex : Type :=  
  | Empty | Eps | Char (a : A)  
  | Alt (r1 r2 : regex) | Seq (r1 r2 : regex)  
  | Star (r : regex) | And (r1 r2 : regex) | Neg (r : regex).
```

A language is a boolean test on words.

```
Definition language := pred word.
```

```
Definition void : language := pred0.
```

```
Definition eps : language := fun w => w == [::].
```

```
Definition atom (a : A) : language := fun w => w == [:: a].
```

```
Definition plus (L K : language) := fun w => L w || K w.
```

```
Definition conc (L K : language) := fun w =>  
  has (fun i => L (take i w) && K (drop i w))  
    (iota 0 (size w).+1).
```

```
Definition star (L : language) := fun w => has (all_in L) (splits w).
```

From expressions to languages

The semantics `den` ties the two together. It sends each constructor to its language operation, so every expression r denotes a language $\mathcal{L}(r)$.

```
Fixpoint den (r : regex) : pred word :=  
  match r with  
  | Empty      => void  
  | Eps        => eps  
  | Char a     => atom a  
  | Alt r1 r2  => plus (den r1) (den r2)  
  | Seq r1 r2  => conc (den r1) (den r2)  
  | Star r1    => star (den r1)  
end.
```

The **restricted** fragment, without $\&$ and $\neg$, is the part the fuzzy layer builds on later.

Does a pattern match the empty word?

We often need to know whether a pattern matches the empty word, the word with no symbols at all. We call it **nullability** ν .

$$\nu(r) = \begin{cases} \varepsilon & \text{if } \varepsilon \in \mathcal{L}(r), \\ \emptyset & \text{otherwise.} \end{cases}$$

```
Fixpoint has_eps (r : regex) : bool :=  
  match r with  
  | Empty      => false  
  | Eps        => true  
  | Char _     => false  
  | Alt r1 r2  => has_eps r1 || has_eps r2  
  | Seq r1 r2  => has_eps r1 && has_eps r2  
  | Star _     => true  
  end.
```

```
Lemma nullable_correct : forall r, has_eps r = ([::] \in r).
```

The derivative $\mathcal{D}_a$

The derivative of a language $L \subseteq \Sigma^*$ with respect to a symbol a removes a leading a ,

$$\partial_a(L) = \{ w \in \Sigma^* \mid a \cdot w \in L \}.$$

On expressions the same operation is defined by structural recursion,

$$\begin{aligned} \mathcal{D}_a(\emptyset) &= \emptyset, & \mathcal{D}_a(\varepsilon) &= \emptyset, \\ \mathcal{D}_a(a) &= \varepsilon, & \mathcal{D}_a(b) &= \emptyset \quad (a \neq b), \\ \mathcal{D}_a(r + s) &= \mathcal{D}_a(r) + \mathcal{D}_a(s), & \mathcal{D}_a(r^*) &= \mathcal{D}_a(r) \cdot r^*, \\ \mathcal{D}_a(r \cdot s) &= \mathcal{D}_a(r) \cdot s + \nu(r) \cdot \mathcal{D}_a(s). \end{aligned}$$

For words, $\mathcal{D}_\varepsilon(r) = r$ and $\mathcal{D}_{aw}(r) = \mathcal{D}_w(\mathcal{D}_a(r))$.

Theorem (Semantic–syntactic correspondence)

For every r and word u , $\mathcal{L}(\mathcal{D}_u(r)) = \partial_u(\mathcal{L}(r))$.

Canonicalization

A **normaliser** rewrites every expression to one standard form $[r] = \text{nf}(r)$, so similar expressions become equal. Each operator has its own rules.

- **Plus** $+$. Flatten nested $+$, drop $\emptyset$, sort the terms, remove duplicates.
- **Concatenation** $\cdot$. Flatten, drop ε , and collapse to $\emptyset$ if any factor is $\emptyset$.
- **Star** $*$. $\emptyset^* = \varepsilon$, $\varepsilon^* = \varepsilon$, $(r^*)^* = r^*$.

Example $r = ((a^*)^* \cdot \varepsilon) + (b + (a + \emptyset)) + b$.

- Subterm: $(a^*)^* \cdot \varepsilon \rightarrow a^*$.
- Plus terms, giving $[r] = a + b + a^*$:

$\{a^*, b, a, \emptyset, b\}$	$\rightarrow \{a^*, b, a, b\}$	drop $\emptyset$
	$\rightarrow \{a, b, b, a^*\}$	sort
	$\rightarrow \{a, b, a^*\}$	dedup

```
Fixpoint plus_terms (r : regex) : seq regex :=
  match r with
  | Alt r1 r2 => plus_terms r1 ++ plus_terms r2
  | _ => [:: r]
end.

Definition mkPlus (r s : regex) : regex :=
  let xs := plus_terms r ++ plus_terms s in
  (* drop empty *)
  let xs := filter (fun t => ~~ isEmpty t) xs in
  (* sort, dedup *)
  let xs := dedup_adj (sortR xs) in
  mkPlus_list xs.
```

The derivative automaton

Definition (Derivative DFA)

For a regular expression r , take

$$Q = \{ [\mathcal{D}_u(r)] \mid u \in \Sigma^* \}, \quad q_0 = [r], \quad \delta(q, a) = [\mathcal{D}_a(q)], \quad F = \{ q : \nu(q) = \varepsilon \}.$$

- The states are the canonical derivatives, each transition takes a derivative, and a state accepts when it matches the empty word.
- We build it by exploring from q_0 : differentiate, canonicalize, and add every new state we reach, until nothing new appears.

Theorem (DFA correctness)

$$\mathcal{L}(M) = \mathcal{L}(r).$$

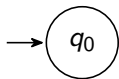
Building the automaton, step by step

```
dfa(r):
  q0      := [r]          (* canonical start *)
  Q       := { q0 }
  delta := empty
  for each a in Sigma do
    goto(q0, a)
  F := { q in Q | nullable q }

goto(q, a):
  qa := [ D_a(q) ]      (* deriv + canon *)
  if qa in Q then
    delta[q,a] := qa    (* already seen: stop *)
  else
    Q          += qa    (* new state *)
    delta[q,a] := qa
    for each a' in Sigma do
      goto(qa, a')      (* recurse *)
```

- 1 Start from the canonical form $q_0 = [r]$, with an empty transition table.
- 2 For each symbol a , call $\text{goto}(q_0, a)$.
- 3 Inside $\text{goto}(q, a)$, differentiate and canonicalize: $q_a = [\mathcal{D}_a(q)]$.
- 4 If q_a is **already known**, just record $q \xrightarrow{a} q_a$ and return. The visited set stops the recursion here.
- 5 If q_a is **new**, add it, record the edge, and recurse into $\text{goto}(q_a, a')$ for every symbol. The search goes depth-first into q_a before coming back up.
- 6 When every branch lands on a known state, the recursion unwinds and the nullable states are marked accepting.

Constructing the automaton of $r = (a+b)^* \cdot a$



Step 1 — Initialize.

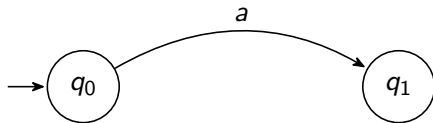
The start state is the canonical form of r ,

$$q_0 = [(a+b)^* \cdot a].$$

Known states: $\{q_0\}$. Now call goto on q_0 for each symbol.

	a	b	acc.
q_0			

Constructing the automaton of $r = (a+b)^* \cdot a$



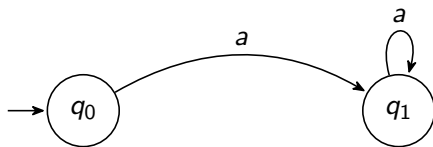
Step 2 — goto(q_0, a)

$$\begin{aligned}\mathcal{D}_a((a+b)^* \cdot a) &= \mathcal{D}_a((a+b)^*) \cdot a + \nu((a+b)^*) \cdot \mathcal{D}_a(a) \\ &= (\mathcal{D}_a(a+b) \cdot (a+b)^*) \cdot a + \varepsilon \cdot \varepsilon \\ &= ((\varepsilon + \emptyset) \cdot (a+b)^*) \cdot a + \varepsilon \cdot \varepsilon \\ &\sim (a+b)^* \cdot a + \varepsilon\end{aligned}$$

A **new** canonical form. Add it as q_1 , record $q_0 \xrightarrow{a} q_1$, and **recurse into** q_1 .

	a	b	acc.
q_0	q_1		
q_1			

Constructing the automaton of $r = (a+b)^* \cdot a$



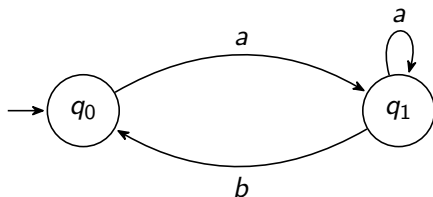
Step 3 — goto(q_1, a), $q_1 = (a+b)^* a + \varepsilon$

$$\begin{aligned}\mathcal{D}_a(q_1) &= \mathcal{D}_a((a+b)^* a) + \mathcal{D}_a(\varepsilon) \\ &\sim q_1 + \emptyset \sim q_1\end{aligned}$$

Already a known state. Record the self-loop $q_1 \xrightarrow{a} q_1$ and return. **The recursion stops here.**

	a	b	acc.
q_0	q_1		
q_1	q_1		

Constructing the automaton of $r = (a+b)^* \cdot a$



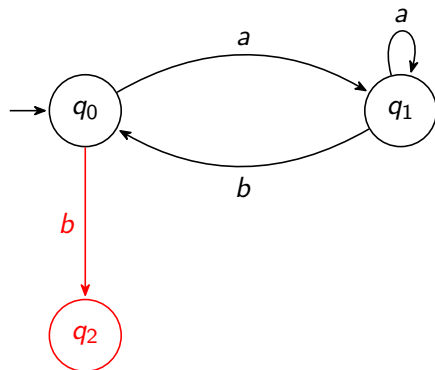
Step 4 — goto(q_1, b)

$$\begin{aligned}\mathcal{D}_b(q_1) &= \mathcal{D}_b((a+b)^*a) + \mathcal{D}_b(\varepsilon) \\ &\sim q_0 + \emptyset \sim q_0\end{aligned}$$

Already known. Record $q_1 \xrightarrow{b} q_0$ and return. q_1 is done, so we climb back up to q_0 .

	a	b	acc.
q_0	q_1		
q_1	q_1	q_0	

Constructing the automaton of $r = (a+b)^* \cdot a$



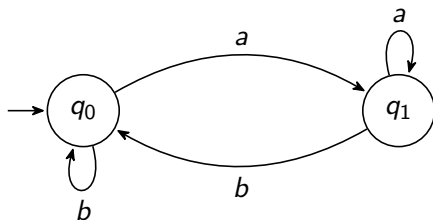
Step 5 — goto(q_0, b)

$$\begin{aligned}\mathcal{D}_b((a+b)^* \cdot a) &= \mathcal{D}_b((a+b)^*) \cdot a + \nu((a+b)^*) \cdot \mathcal{D}_b(a) \\ &= (\mathcal{D}_b(a+b) \cdot (a+b)^*) \cdot a + \varepsilon \cdot \emptyset \\ &= ((\emptyset + \varepsilon) \cdot (a+b)^*) \cdot a + \varepsilon \cdot \emptyset\end{aligned}$$

Left as is, this is a **brand new state** q_2 (in red). But if we canonicalize it, this state turns out to be the same as q_0 , so we add no new state.

	a	b	acc.
q_0	q_1		
q_1	q_1	q_0	

Constructing the automaton of $r = (a+b)^* \cdot a$



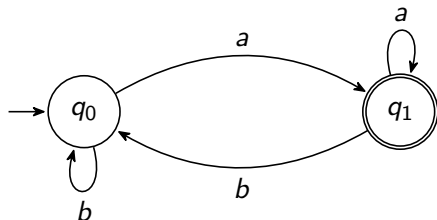
Step 5 — canonicalize

$$((\emptyset + \varepsilon) \cdot (a+b)^*) \cdot a + \varepsilon \cdot \emptyset \sim (a+b)^* \cdot a = q_0$$

This state **collapses into** q_0 . We record the self-loop $q_0 \xrightarrow{b} q_0$ instead of adding a state.

	a	b	acc.
q_0	q_1	q_0	
q_1	q_1	q_0	

Constructing the automaton of $r = (a+b)^* \cdot a$



Step 6 — accepting states and halt

$$q_0 = (a+b)^* \cdot a, \quad q_1 = (a+b)^* \cdot a + \varepsilon.$$

$$\nu(q_0) = \emptyset \Rightarrow \text{no}, \quad \nu(q_1) = \varepsilon \Rightarrow \text{yes}$$

No new canonical form appears, so the construction **halts**.
From $r = (a+b)^* \cdot a$ we have built a two-state DFA that accepts the words over $\{a, b\}$ ending in a .

	a	b	acc.
q_0	q_1	q_0	no
q_1	q_1	q_0	yes

Where we are

- 1 Regular expressions and derivatives
- 2 Rocq
- 3 Automata
- 4 Fuzzy layer**
- 5 Conclusion

When exact matching is too strict

Exact matching asks whether two symbols are the very same symbol. Often that is too strict. Text is noisy and symbols get mistyped or swapped for close ones. We would like a pattern to keep matching when the word is only close to it, and derivatives should follow the same rule.

Where matching up to closeness helps:

- A **search query** still finds words that differ from it by a few similar letters.
- In **biological sequences** bases or amino acids that are chemically close count as near matches.
- In **text from OCR or speech** each symbol is read with some confidence, so near misses are common and should still match.

The plan

We keep the whole derivative construction. We change only what it means for two symbols to match. Everything else follows from that one change.

First we need a similarity relation

To match symbols by closeness we first fix a relation that grades how similar two symbols are. It gives each pair a degree between 0 and 1. A **proximity** relation is reflexive and symmetric. A **similarity** relation adds transitivity through a T-norm, and we use the minimum. A threshold μ , the **cut**, turns a degree back into a plain yes or no.

```
Definition fuzzy_rel := S -> S -> simval.    (* a degree in [0,1] *)
```

```
(* the cut keeps only the pairs that are similar enough *)
```

```
Definition mu_cut (mu : simval) (R : fuzzy_rel) : S -> S -> Prop :=  
  fun s1 s2 => mu <= R s1 s2.
```

```
Record is_proximity (R : fuzzy_rel) : Prop := {  
  prox_range : is_fuzzy_rel R;  
  prox_refl  : forall s, R s s = 1;  
  prox_sym   : forall s1 s2, R s1 s2 = R s2 s1 }.
```

```
Record is_similarity (R : fuzzy_rel) : Prop := {  
  sim_prox   : is_proximity R;  
  sim_trans  : forall s1 s2 s, smin (R s1 s) (R s s2) <= R s1 s2 }.
```

Fuzzy derivatives and what we found

The **fuzzy derivative** is the ordinary derivative with one change. A symbol now matches whenever it is μ -similar to the one we read, not only when it is equal.

Under the minimum T-norm the cut relation is an **equivalence**. So we may group μ -similar symbols into classes and read over the **quotient alphabet** Σ^μ .

Theorem (Quotient form of the fuzzy derivative)

A fuzzy derivative over Σ is an ordinary derivative over the quotient alphabet Σ^μ :

$$q_\mu^*(\partial_a^\mu(\mathcal{L})) = \partial_{[a]_\mu}(\mathcal{L}^\mu).$$

So the whole crisp theory carries over. Fuzzy derivatives stay regular and finite, meaning and computation still agree, and the fuzzy automaton is still correct.

Where we are

- 1 Regular expressions and derivatives
- 2 Rocq
- 3 Automata
- 4 Fuzzy layer
- 5 Conclusion**

- The crisp derivative theory, up to automaton correctness, is fully **checked by Rocq**.
- The new part is the **fuzzy layer**. Under the minimum T-norm, cut-similarity is an equivalence, so a fuzzy derivative is just an ordinary derivative over a quotient alphabet.
- That one idea carries regularity, correspondence, finiteness and fuzzy automaton correctness over to the fuzzy case.
- The next step is to formalize the fuzzy layer in Rocq.

Thank you

Questions?